데이터과학 파이썬

심송용(한림대학교 데이터과학스쿨)

http://jupiter.hallym.ac.kr

numpy 패키지

행렬

행렬은 2차원 ndarray이므로 다음과 같이 생성할 수 있다.

```
A = np.array( [[1,1], [0,1]] )
B = np.array( [[2,0], [3,4]] )
print(A, B)
```

⇨ [[1 1] [[2 0]
  [0 1]] [3 4]]

이 출력에서 ndarray는 각 행별로 출력을 해주어서 리스트 등 파이썬의 기본 배열보다 출력시 가독성이 높음을 알 수 있다.

특정한 위치의 원소는 A[i,j]로 인덱싱이 가능하다. 파이썬의 기본 인덱싱인 A[i][j]로도 같은 위치의 값을 인덱싱할 수 있다.

```
print(A[0,1])
print(A[0][1])
```

모두 1을 인쇄한다.

numpy 패키지

사칙연산 등의 기본 연산자들의 ndarray 연산은 같은 위치의 원소대 원소로 이루어진다. 즉, A+B의 결과는 A[i,j] + B[i,j]이다.

```
print(A * B)
 [[2 0]
  [0 4]]
```

행렬의 곱은 @나 dot 속성으로 계산할 수 있다.

```
print(A @ B)    또는
print(A.dot(B))
```

의 결과는 모두

```
[[5 4]
 [3 4]]
```

이다.

numpy 패키지

모든 원소가 0이나 1인 배열

ndarray.ones와 ndarray.zeros는 각각 모든 원소가 1과 0인 배열을 만들어 준다.

```
numpy.ones(shape, dtype=None, ...)
    또는
numpy.zeros(shape, dtype=None, ...)
```

사용하며 shape에는 정수 또는 정수로 이루어진 배열로 차원을 정의한다.

```
print(np.ones(5)) ⇨ [1. 1. 1. 1. 1.]
print(np.zeros(4))  ⇨ [0. 0. 0. 0.]
J = np.ones([5,5])
print('J: ', J)
J:  [[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
```

numpy 패키지

```
myones = np.ones([5,1])
print('Jx1: ', J @ myones)
print('Jx1: ', J.dot(myones))
```

는 같은 결과

```
Jx1:  [[5.]
 [5.]
 [5.]
 [5.]
 [5.]]
```

를 얻는다.

값이 설정되지 않은 ndarray

값이 설정되지 않은(그러나 실제로는 임의의 값이 들어가 있는) ndarray는 np.empty로 만들 수 있다.

```
numpy.empty(shape, dtype=float, ...)
```

- shape에는 만들어질 ndarray의 차원을 설정하고

numpy 패키지

- dtype은 만들어질 ndarray의 데이터 타입을 설정한다.

```
a1 = np.empty((2,3), dtype=int)
print(a1)
```

```
[[-858993459 1074318540 -858993459]
 [1074580684 1717986918 1074816614]]
```

a1에는 의미없는 값들이 들어가 있다.

ndarray의 차원

ndarray의 차원은 shape 속성으로 알아낼 수 있으며, reshape 메소드를 사용하여 차원을 변경할 수도 있다. reshape 메소드는

```
ndarray.reshape(shape, order='C')
```

로 사용하며 shape에 차원을 설정하며 결과로 차원변경된 ndarray를 반환한다.

```
print(a1.shape)    ⇨  (2, 3)
a2 = a1.reshape(3,2)
print(a2)
```

numpy 패키지

```
[[-858993459 1074318540]
 [-858993459 1074580684]
 [1717986918 1074816614]]
```

print('행의 수: ', a2.shape[0]) ⇨ 행의 수: 3

난수발생기

유한 모집단에서 표본 얻기

주어진 모집단에서 특정 개수의 임의표본을 얻고자 할 때 choice 함수를 사용할 수 있다. 이 함수는

```
numpy.random.choice(pop, size=None, replace=True, p=None)
```

로 사용하며

- pop: 추출할 모집단을 설정하며 1차원 array 또는 자연수 값을 설정한다. 1차원 배열이 설정되면 배열의 원소에서 표본을 얻으며, 자연수가 설정되면 np.arange(a)에서 표본을 뽑는다.
- size는 추출할 개수를 설정하며 설정하지 않으면 1개를 추출하게 된다.
- replace는 복원추출 여부를 설정하며 True이면 복원추출을, False이면 비복원추출을 하

numpy 패키지

게 된다. replace가 False이면 size는 len(a)보다 클 수 없다.

- p: pop의 각 원소의 추출확률을 설정한다. 기본값은 1/len(pop)으로 등확률이다.

```
# numpytest2.py
import numpy as np
import statistics

print(np.random.choice(5))    ⇨ 0
print(np.random.choice(5, size=2))  ⇨ [2 0]
x=['r','g','b','c','m','y','k']
print(np.random.choice(x, size=7, replace=False))  ['k' 'y' 'r' 'c' 'g' 'm' 'b']
```

size와 x의 길이가 같고 replace가 False이면 원소들의 permutation이 얻어진다.

위의 결과 및 아래의 결과들(난수발생) 실행할 때마다 달라짐(그래야 임의표본임)

특정 범위내의 정수에서 임의 추출을 할 경우 randint 메소드를 사용할 수 있다. 이 함수는

numpy 패키지

numpy.random.randint(low, high=None, size=None, dtype='l')

로 사용하며 low부터 high 사이의 정수(low 포함, high 미포함)를 size에 설정한 개수만큼 생성한다(복원 추출).

```
x = np.random.randint(1,11, size=1000).astype(float)
# 위의 float가 없으면 정수로 계산하게 되어 아래 값이 정수부분만 얻어짐에 유의
print('mean:', statistics.mean(x))
print('variance:', statistics.variance(x))
```

mean: 5.422

variance: 8.486402402402403

1부터 10사이의 숫자에서 임의로 1,000개의 난수를 발생하여 이들의 평균과 분산을 계산하였다. 이론적인 값은 $\mu = 5.5$, $\sigma^2 = (385 - 10 \cdot 5.5^2)/10 = 8.25$이므로 1,000개 표본에서 얻은 값은 이 값과 비슷한 값을 얻을 것임(실행할 때마다 결과가 다름)

ndarray의 데이터 타입은 astype 속성으로 바꿀 수 있다.